CHAPTER 7: ADVANCED SQL

Modern Database Management
11th Edition, International Edition

Jeffrey A. Hoffer, V. Ramesh, Heikki Topi

OBJECTIVES

- Define terms
- Write single and multiple table SQL queries
- Define and use three types of joins
- Write noncorrelated and correlated subqueries
- Understand and use SQL in procedural languages (e.g. PHP, PL/SQL)
- Understand triggers and stored procedures
- Discuss SQL:2008 standard and its enhancements and extensions

PROCESSING MULTIPLE TABLES

- Join–a relational operation that causes two or more tables with common columns to be combined into a single table or view
- Equi-join–a join in which the joining condition is based on equality between values in the common columns; common columns appear twice in the result table
- Natural join–an equi-join in which one of the duplicate columns is eliminated in the result table

Example

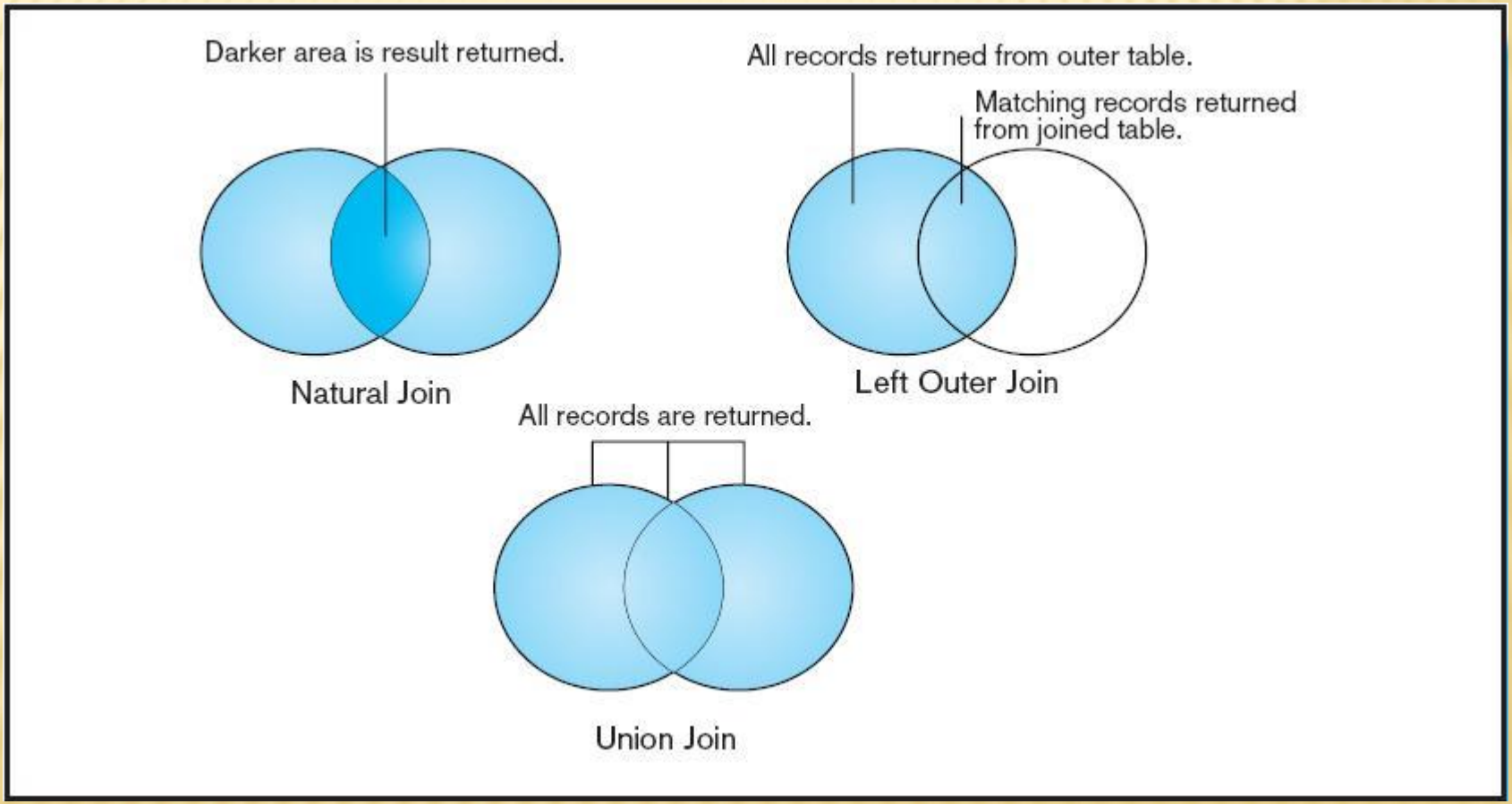
The common columns in joined tables are usually the primary key of the dominant (parent) table and the foreign key of the dependent (child) table in 1:M relationships.

PROCESSING MULTIPLE TABLES

- Outer join
  - In joining two tables, a row in one table may not have a matching row in the other table
  - A join in which rows that do not have matching values in common columns are also included in the result table
  - As opposed to *inner* join, in which rows must have matching values in order to appear in the result table
- Union join

  Includes all columns from the two joined tables, and an instance for each row of each table

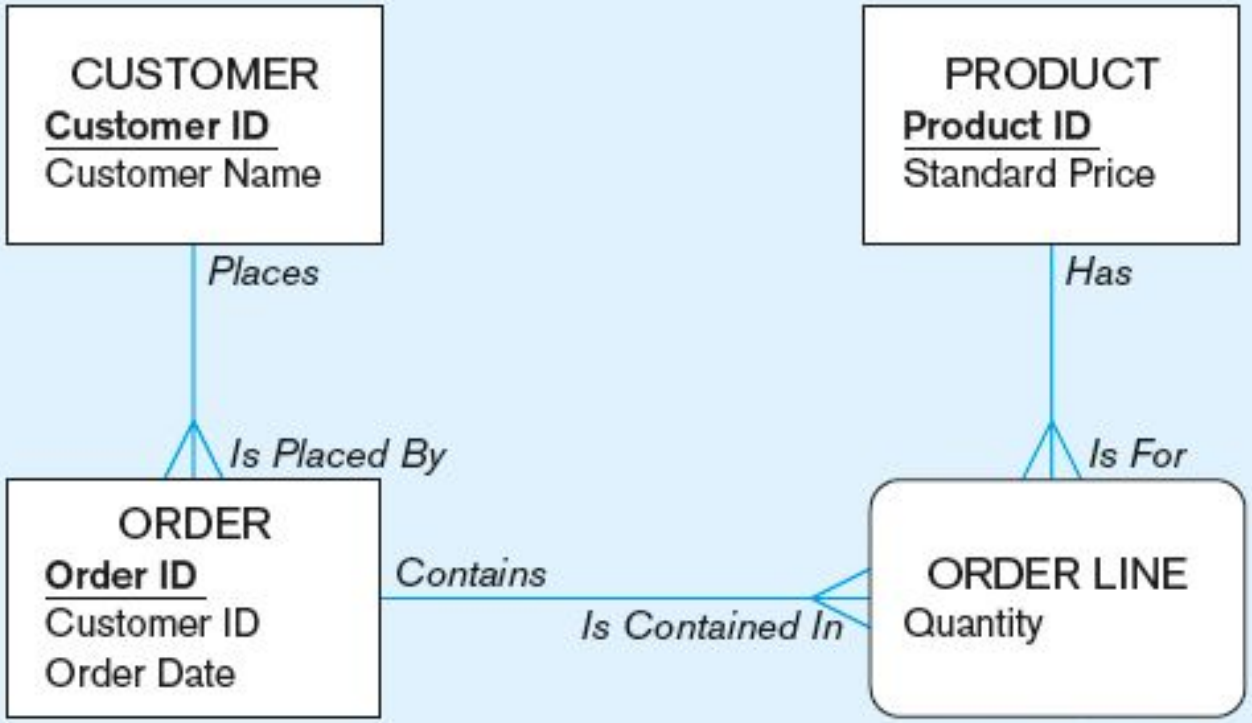
Figure 7-2
Visualization of different join types with results returned in shaded area



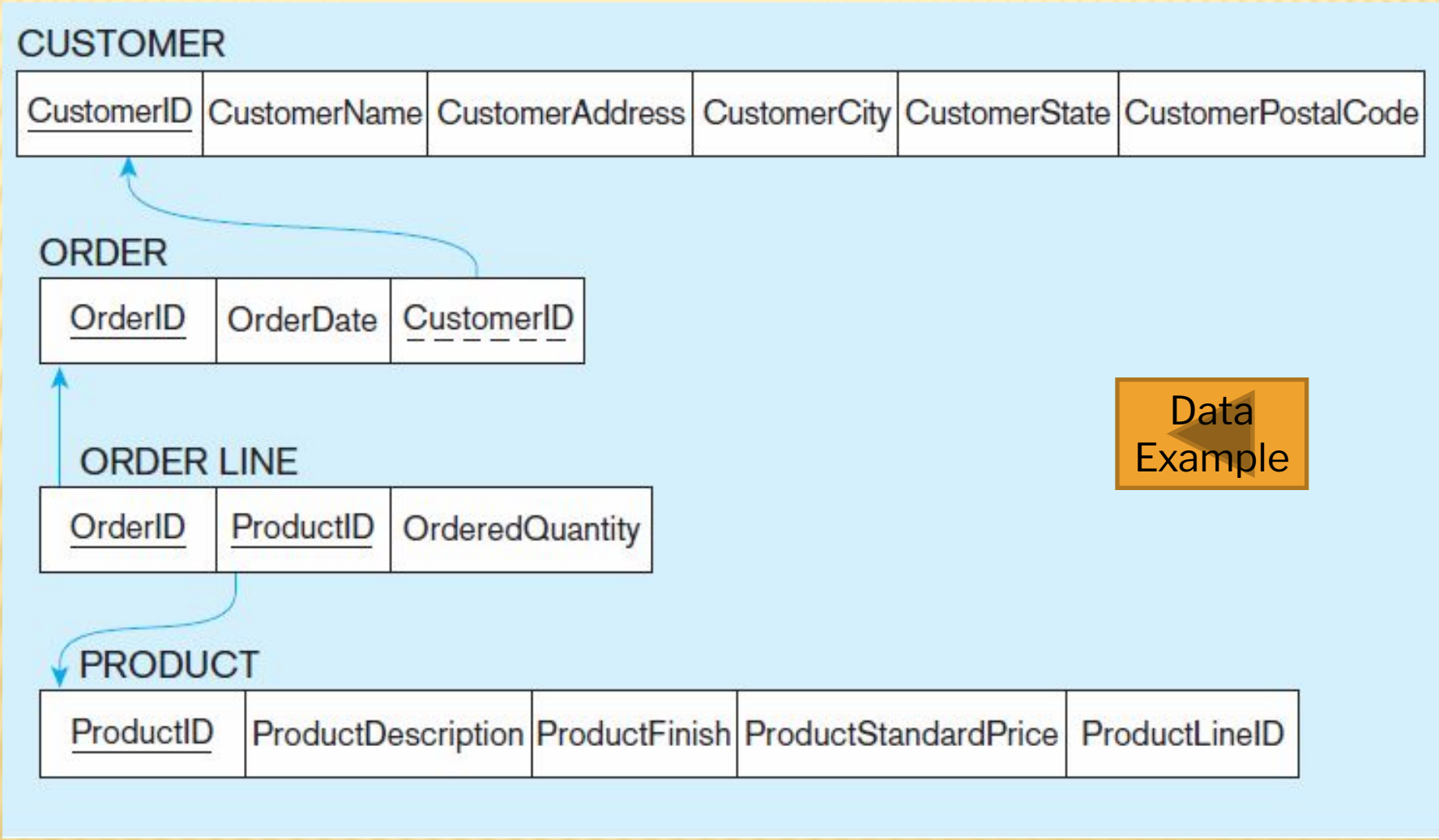
THE FOLLOWING SLIDES CREATE TABLES FOR THIS ENTERPRISE DATA MODEL

(from Chapter 1, Figure 1-3)

Corresponding Relational Schema

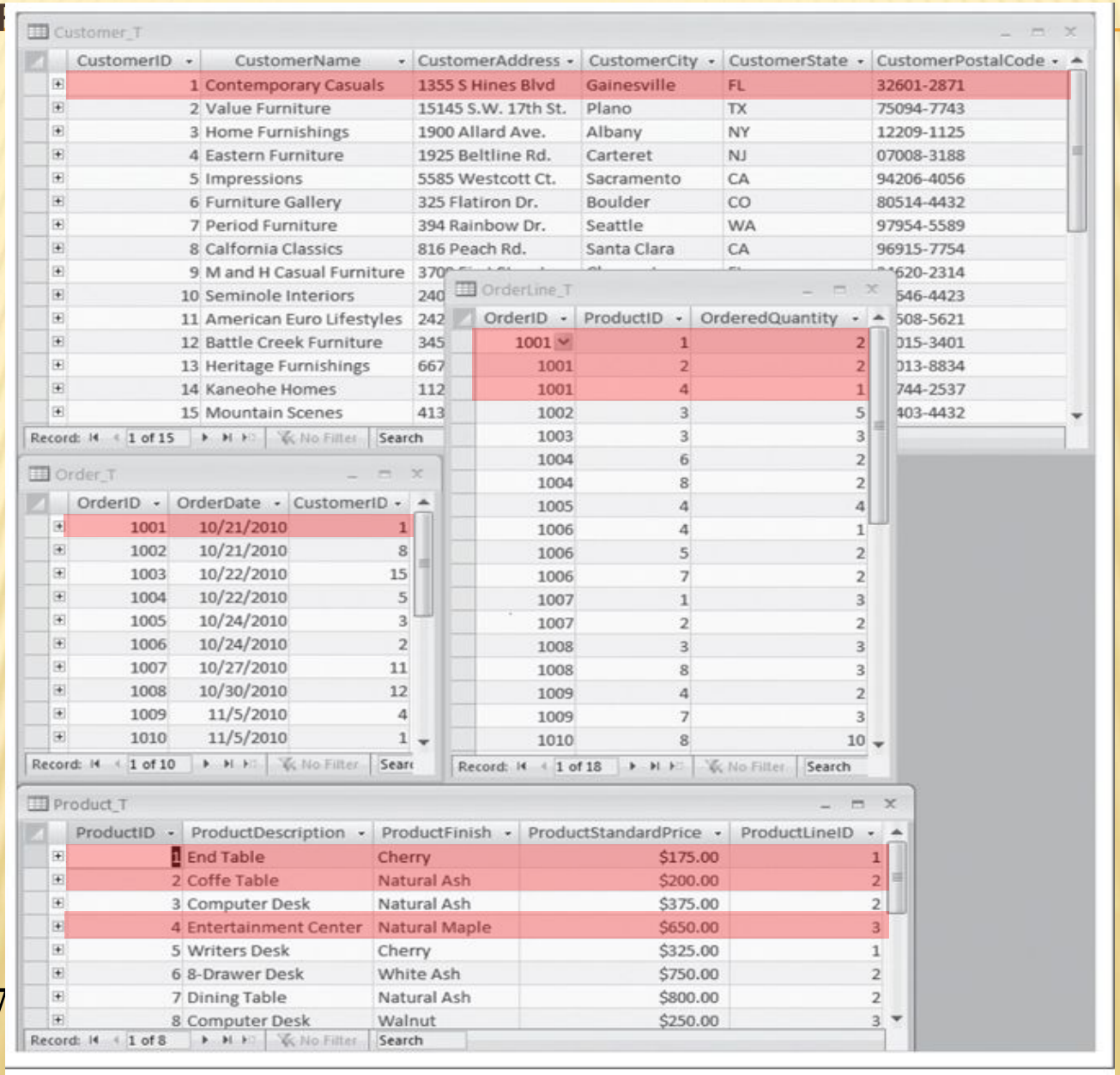


(From Figure 4-5 Relational Schema for Pine Valley Furniture)



FROM FIGURE 6.3 : SAMPLE DATA OF A RELATIONAL DATABASE

(DATA FILES AT FTP://163.25.117.117 → CCHEN→102FALL →102F_DBMS →DATA F



Customer_T

CustomerID	CustomerName	CustomerAddress	CustomerCity	CustomerState	CustomerPostalCode
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094-7743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	12209-1125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	94206-4056
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
7	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
8	Calfornia Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
9	M and H Casual Furniture	370			620-2314
10	Seminole Interiors	240			546-4423
11	American Euro Lifestyles	242			508-5621
12	Battle Creek Furniture	345			015-3401
13	Heritage Furnishings	667			013-8834
14	Kaneohe Homes	112			744-2537
15	Mountain Scenes	413			403-4432

Record: 1 of 15 | No Filter | Search

OrderLine_T

OrderID	ProductID	OrderedQuantity
1001	1	2
1001	2	2
1001	4	1
1002	3	5
1003	3	3
1004	6	2
1004	8	2
1005	4	4
1006	4	1
1006	5	2
1006	7	2
1007	1	3
1007	2	2
1008	3	3
1008	8	3
1009	4	2
1009	7	3
1010	8	10

Record: 1 of 18 | No Filter | Search

Order_T

OrderID	OrderDate	CustomerID
1001	10/21/2010	1
1002	10/21/2010	8
1003	10/22/2010	15
1004	10/22/2010	5
1005	10/24/2010	3
1006	10/24/2010	2
1007	10/27/2010	11
1008	10/30/2010	12
1009	11/5/2010	4
1010	11/5/2010	1

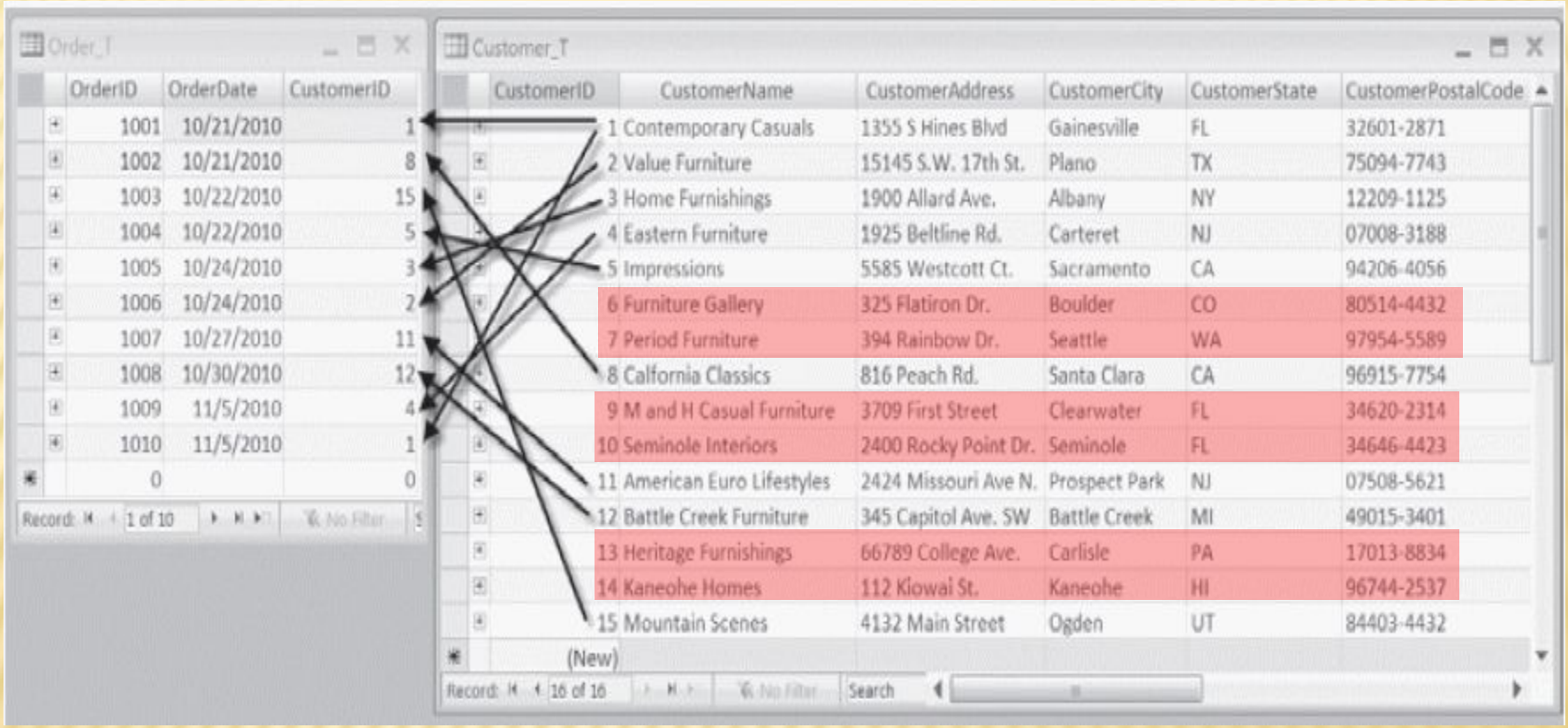
Record: 1 of 10 | No Filter | Sear

Product_T

ProductID	ProductDescription	ProductFinish	ProductStandardPrice	ProductLineID
1	End Table	Cherry	$175.00	1
2	Coffe Table	Natural Ash	$200.00	2
3	Computer Desk	Natural Ash	$375.00	2
4	Entertainment Center	Natural Maple	$650.00	3
5	Writers Desk	Cherry	$325.00	1
6	8-Drawer Desk	White Ash	$750.00	2
7	Dining Table	Natural Ash	$800.00	2
8	Computer Desk	Walnut	$250.00	3

Record: 1 of 8 | No Filter | Search

Figure 7-1 Pine Valley Furniture Company Customer_T and Order_T tables with pointers from customers to their orders



Order_T

OrderID	OrderDate	CustomerID
1001	10/21/2010	1
1002	10/21/2010	8
1003	10/22/2010	15
1004	10/22/2010	5
1005	10/24/2010	3
1006	10/24/2010	2
1007	10/27/2010	11
1008	10/30/2010	12
1009	11/5/2010	4
1010	11/5/2010	1
0		0

Record: 1 of 10

Customer_T

CustomerID	CustomerName	CustomerAddress	CustomerCity	CustomerState	CustomerPostalCode
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094-7743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	12209-1125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	94206-4056
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
7	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
8	Calfornia Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
9	M and H Casual Furniture	3709 First Street	Clearwater	FL	34620-2314
10	Seminole Interiors	2400 Rocky Point Dr.	Seminole	FL	34646-4423
11	American Euro Lifestyles	2424 Missouri Ave N.	Prospect Park	NJ	07508-5621
12	Battle Creek Furniture	345 Capitol Ave. SW	Battle Creek	MI	49015-3401
13	Heritage Furnishings	66789 College Ave.	Carlisle	PA	17013-8834
14	Kaneohe Homes	112 Kiowai St.	Kaneohe	HI	96744-2537
15	Mountain Scenes	4132 Main Street	Ogden	UT	84403-4432
(New)					

Record: 16 of 16 Search

These tables are used in queries that follow

EQUI-JOIN EXAMPLE

- **Query** : What are the IDs and names of all customers, along with the order IDs for all the orders they have placed?

```
SELECT Customer_T.CustomerID, Order_T.CustomerID,
   CustomerName, OrderID
      FROM Customer_T, Order_T
         WHERE Customer_T.CustomerID = Order_T. CustomerID
         ORDER BY OrderID
```

Result:

CUSTOMERID	CUSTOMERID	CUSTOMERNAME	ORDERID
1	1	Contemporary Casuals	1001
8	8	California Classics	1002
15	15	Mountain Scenes	1003
5	5	Impressions	1004
3	3	Home Furnishings	1005
2	2	Value Furniture	1006
11	11	American Euro Lifestyles	1007
12	12	Battle Creek Furniture	1008
4	4	Eastern Furniture	1009
1	1	Contemporary Casuals	1010

10 rows selected.

Tables & Data

Note : Customer ID appears twice in the result

EQUI-JOIN EXAMPLE – ALTERNATIVE SYNTAX

```
SELECT Customer_T.CustomerID, Order_T.CustomerID,
   CustomerName, OrderID
FROM Customer_T INNER JOIN Order_T ON
   Customer_T.CustomerID = Order_T.CustomerID
ORDER BY OrderID;
```

- This query produces same results as previous equi-join example.
- INNER JOIN clause is an alternative to WHERE clause, and it is used to match primary and foreign keys.
- An INNER JOIN will only return rows from each table that have matching rows in the other.

NATURAL JOIN EXAMPLE

- **Query** : What are the IDs and names of all customers, along with the order IDs for all the orders they have placed?

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM Customer_T NATURAL JOIN Order_T ON
Customer_T.CustomerID = Order_T.CustomerID;
```

ON clause performs the equality check for common columns of the two tables

Note: From Fig. 7-1, you see that only 10 Customers have links with orders.

- Only 10 rows will be returned from this join

LEFT OUTER JOIN EXAMP

Venn Diagram

- **Query** : List customer name, ID number, and order number for all customers listed in the CUSTOMER table. Include the customer ID number and name even if that customer hasn't placed an order.

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
   FROM Customer_T LEFT OUTER JOIN Order_T
   WHERE Customer_T.CustomerID = Order_T. CustomerID;
```

LEFT OUTER JOIN causes customer data to appear even if there is no corresponding order data

Unlike INNER join, this will include customer rows with no matching order rows

Left Outer Join Result

customer rows with no matching order rows

CUSTOMERID	CUSTOMERNAME	ORDERID
1	Contemporary Casuals	1001
1	Contemporary Casuals	1010
2	Value Furniture	1006
3	Home Furnishings	1005
4	Eastern Furniture	1009
5	Impressions	1004
6	Furniture Gallery	
7	Period Furniture	
8	California Classics	1002
9	M & H Casual Furniture	
10	Seminole Interiors	
11	American Euro Lifestyles	1007
12	Battle Creek Furniture	1008
13	Heritage Furnishings	
14	Kaneohe Homes	
15	Mountain Scenes	1003

16 rows selected.

MULTIPLE TABLE JOIN EXAMPLE

- **Query**: Assemble all information necessary to create an invoice for order nu

Relational schema

```
SELECT Customer_T.CustomerID, CustomerName, CustomerAddress,
   CustomerCity, CustomerState, CustomerPostalCode, Order_T.OrderID,
   OrderDate, OrderedQuantity, ProductDescription, StandardPrice,
   (OrderedQuantity * ProductStandardPrice)
FROM Customer_T, Order_T, OrderLine_T, Product_T
   WHERE Order_T.CustomerID = Customer_T.CustomerID
      AND Order_T.OrderID = OrderLine_T.OrderID
      AND OrderLine_T.ProductID = Product_T.ProductID
      AND Order_T.OrderID = 1006;
```

Four tables involved in this join

Linking Example

Result

Each pair of tables requires an equality-check condition in the WHERE clause, matching primary keys against foreign keys.

DIAGRAM DEPICTING A FOUR-TABLE JOIN

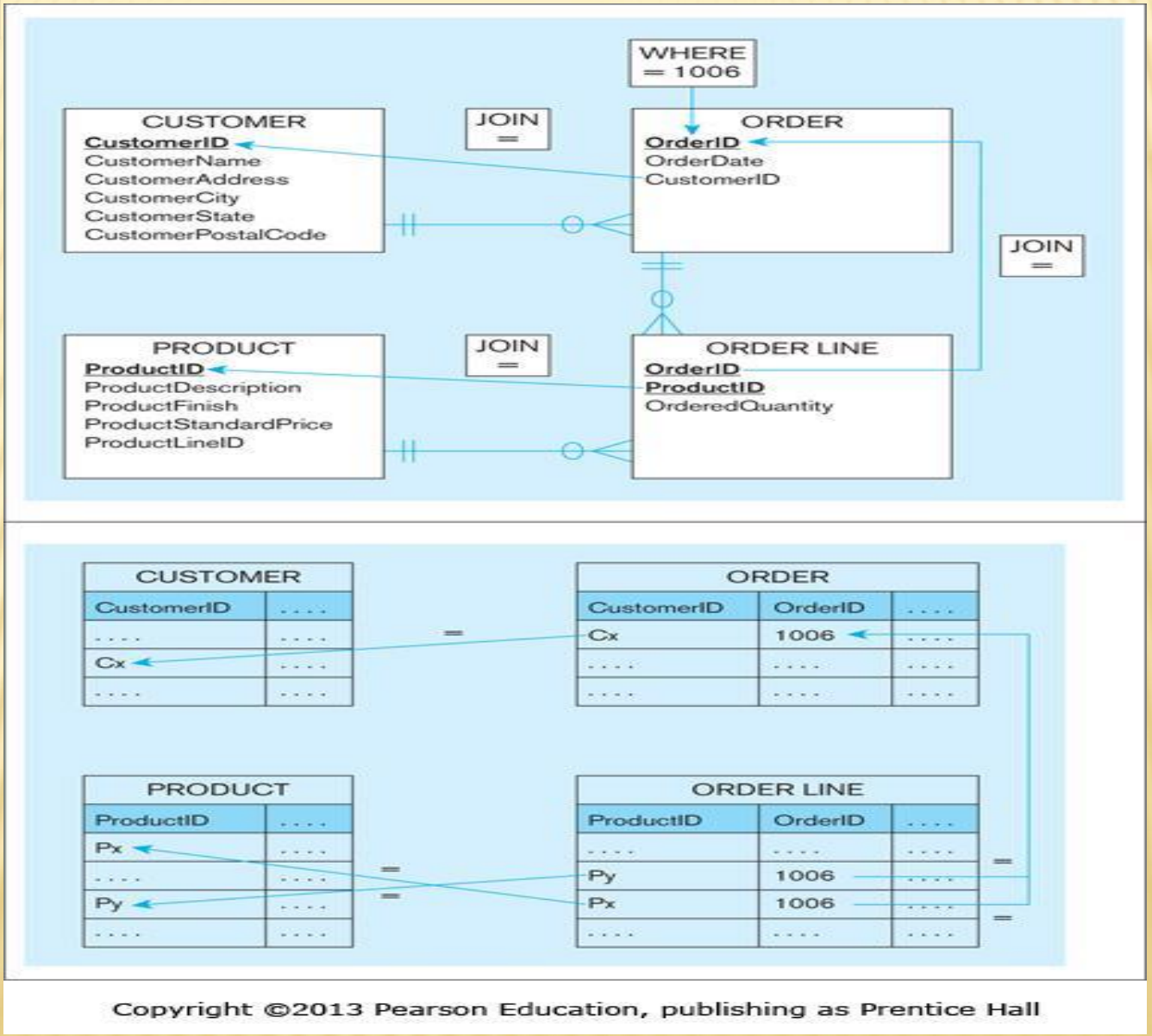


Figure 7-4 Results from a four-table join (edited for readability)

Data

From CUSTOMER_T table

CUSTOMERID	CUSTOMERNAME	CUSTOMERADDRESS	CUSTOMER CITY	CUSTOMER STATE	CUSTOMER POSTALCODE
2	Value Furniture	15145 S. W. 17th St.	Plano	TX	75094 7743
2	Value Furniture	15145 S. W. 17th St.	Plano	TX	75094 7743
2	Value Furniture	15145 S. W. 17th St.	Plano	TX	75094 7743

ORDERID	ORDERDATE	ORDERED QUANTITY	PRODUCTNAME	PRODUCT STANDARDPRICE	(QUANTITY* STANDARDPRICE)
1006	24-OCT -10	1	Entertainment Center	650	650
1006	24-OCT -10	2	Writer's Desk	325	650
1006	24-OCT -10	2	Dining Table	800	1600

From Order_T

From Product_T table

From OrderLine_T

SELF-JOIN EXAMPLE

Query: What are the employee ID and name of each employee and the name of his or her supervisor (label the supervisor's name Manager)? Data Example

```
SELECT E.EmployeeID, E.EmployeeName, M.EmployeeName AS Manager
  FROM Employee_T E, Employee_T M
  WHERE E.EmployeeSupervisor = M.EmployeeID;
```

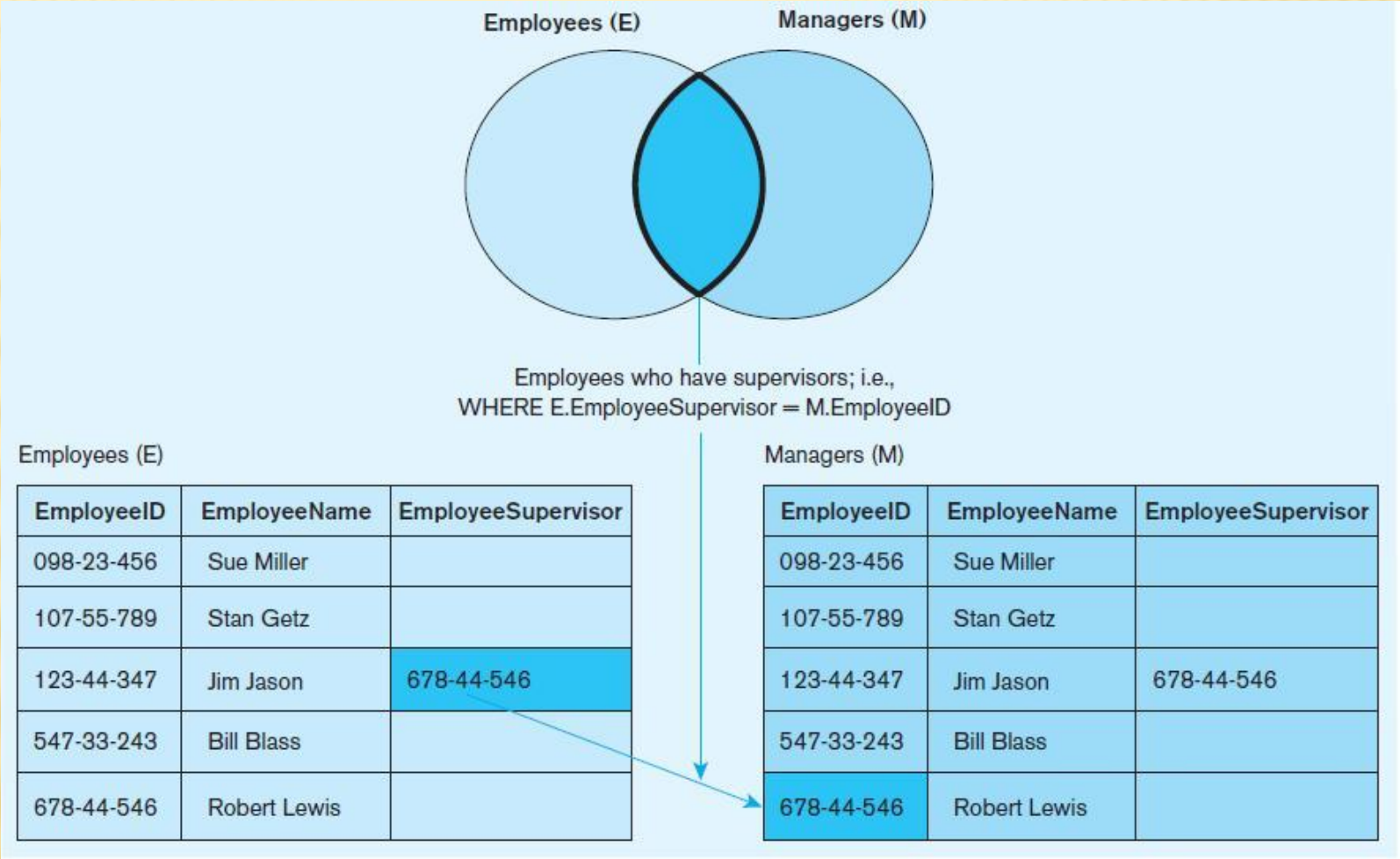
Result:

EMPLOYEEID	EMPLOYEENAME	MANAGER
123-44-347	Jim Jason	Robert Lewis

The same table is used on both sides of the join; distinguished by using table aliases

Self-joins are usually used on tables with unary relationships.

Figure 7-5 Example of a self-join



Employees (E)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

Managers (M)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

PROCESSING MULTIPLE TABLES USING SUBQUERIES

- Subquery–placing an inner query (SELECT statement) inside an outer query
- Options:
  - In a condition of the WHERE clause
  - Within the HAVING clause (in a condition for groups)
  - As a "table" of the FROM clause
- Subqueries can be:
  - Noncorrelated–executed once for the entire outer query
  - Correlated–executed once for each row processed by the outer query

EXAMPLE OF SUBQUERY IN WHERE CLAUSE

- Query: What are the names of customers who have placed or

Data Example

```
SELECT CustomerName
    FROM Customer_T
        WHERE CustomerID IN
            (SELECT DISTINCT CustomerID
            FROM Order_T);
```

The IN operator checks if the CUSTOMER_ID value of a data row is in the list returned from the subquery

Subquery is embedded in parentheses.
It returns a list that will be used in the WHERE clause of the outer query

Result:

CUSTOMER_NAME
Contemporary Casuals
Value Furniture
Home Furnishings
Eastern Furniture
Impressions
California Classics
American Euro Lifestyles
Battle Creek Furniture
Mountain Scenes

9 rows selected.

JOIN VS. SUBQUERY

- Some queries could be accomplished by either a join or a subquery

Query: What are the name and address of the customer who placed order number 1008?

Data Example

```
SELECT CustomerName, CustomerAddress, CustomerCity,
   CustomerState, CustomerPostalCode
FROM Customer_T, Order_T
WHERE Customer_T.CustomerID = Order_T. CustomerID
   AND OrderID = 1008;
```

Join version

Graphical illustration

Subquery version

Graphical illustration

```
SELECT CustomerName, CustomerAddress, CustomerCity,
CustomerState, CustomerPostalCode
   FROM Customer_T
      WHERE Customer_T.CustomerID =
         (SELECT Order_T.CustomerID
            FROM Order_T
               WHERE OrderID = 1008);
```

Figure 7-6 (a) : Graphical depiction of two ways to answer a query with different types of joins

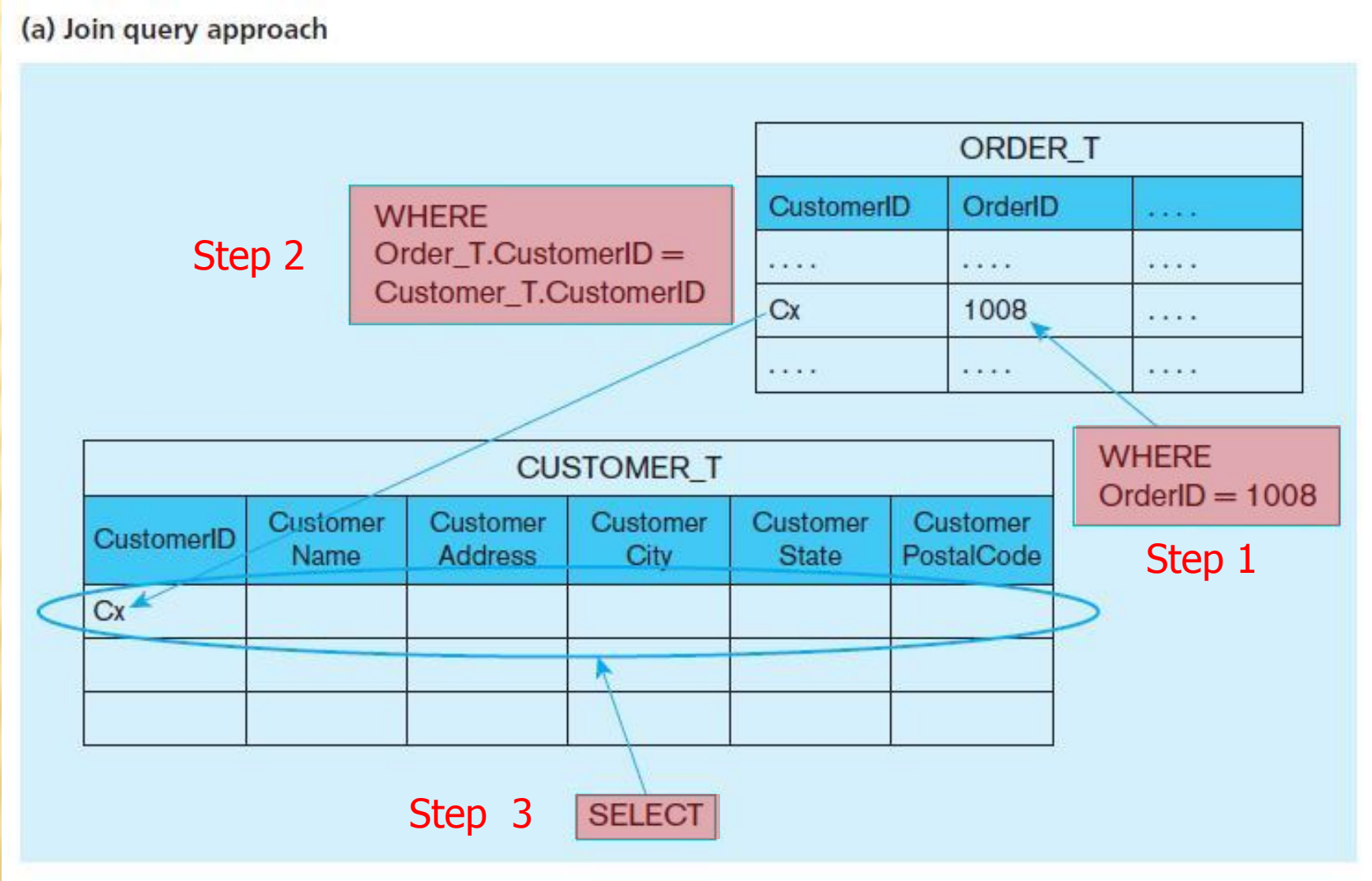
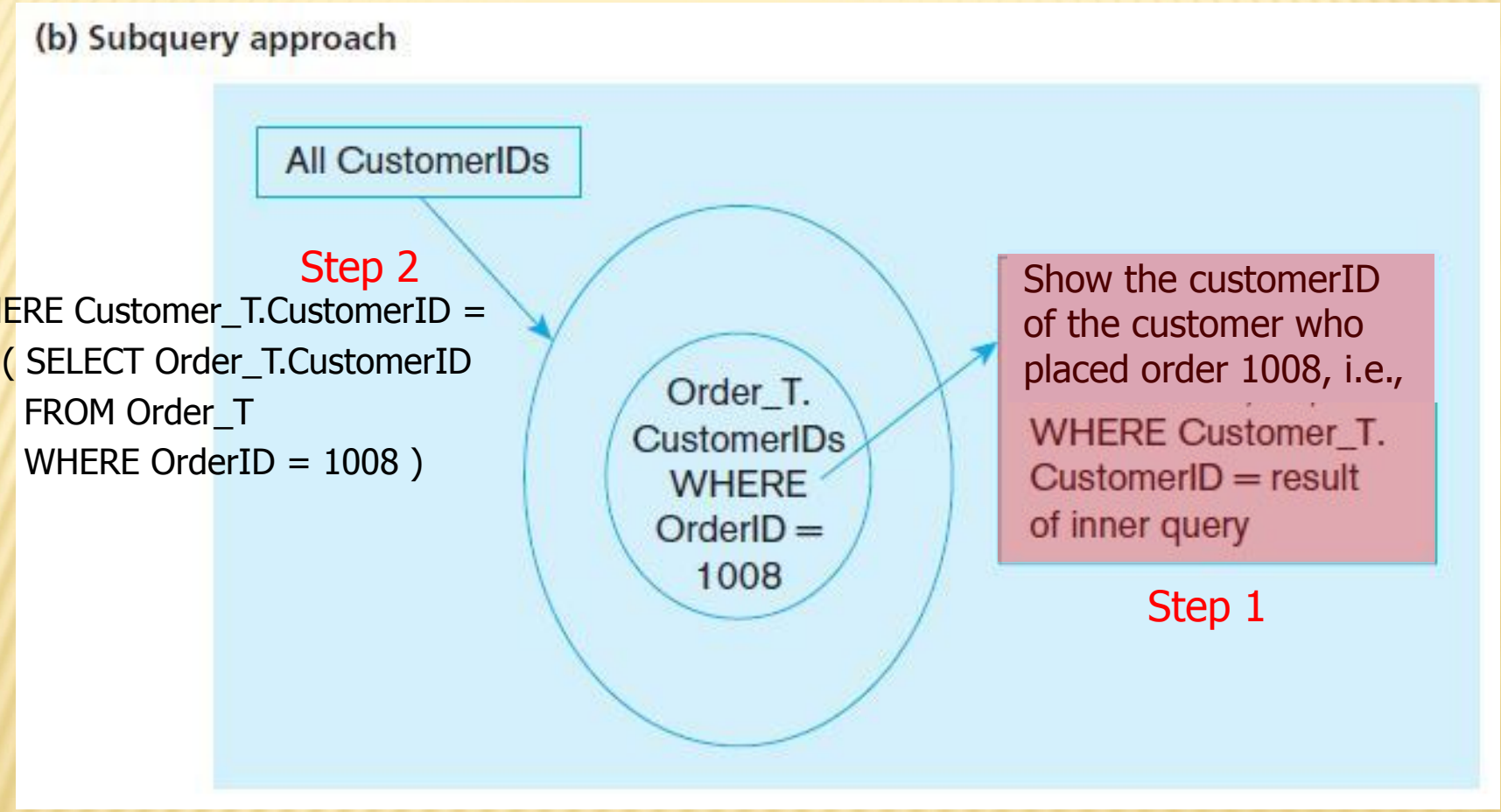


Figure 7-6 (b) : Graphical depiction of two ways to answer a query with different types of joins

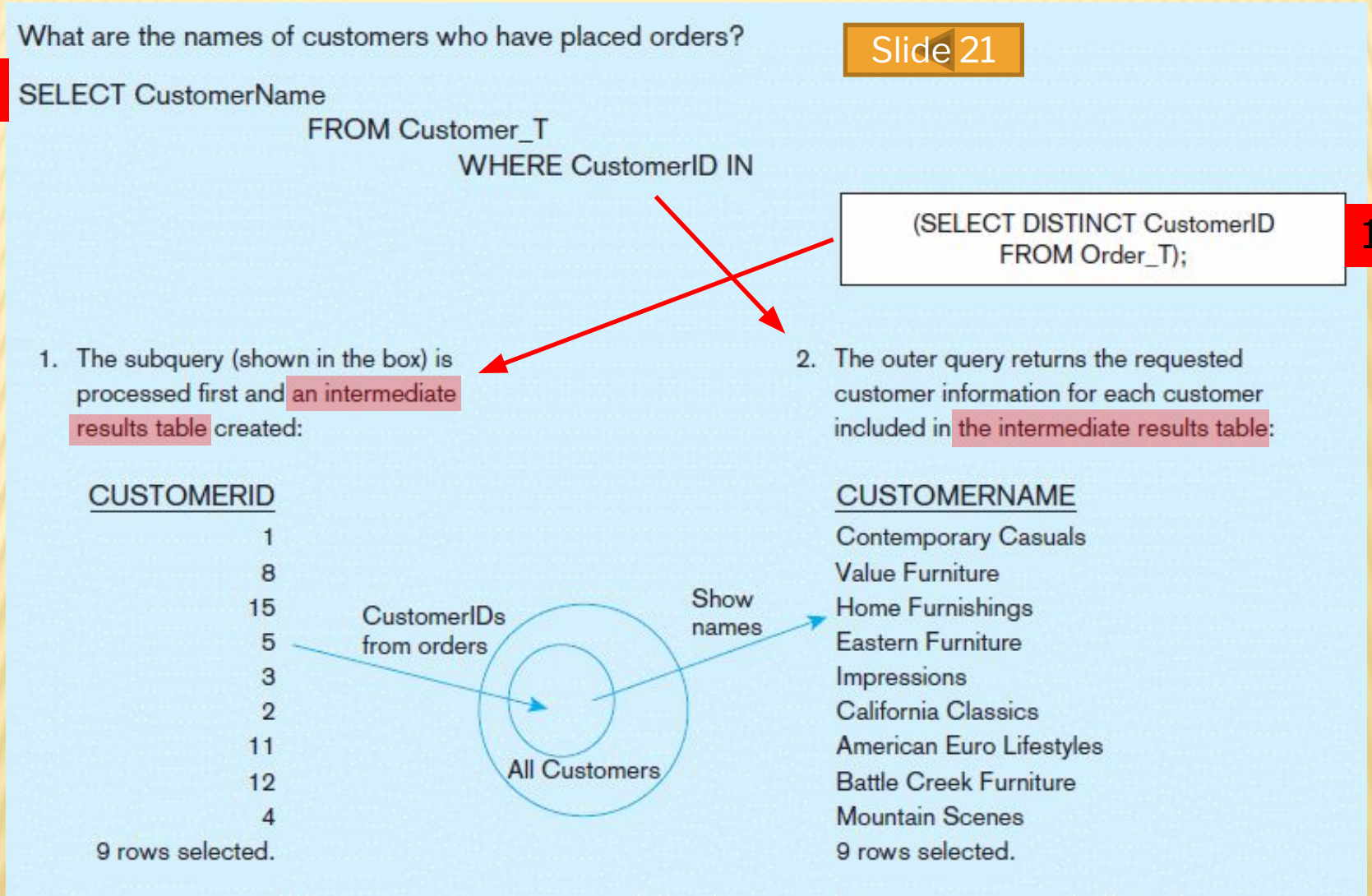


CORRELATED VS. NONCORRELATED SUBQUERIES

- Noncorrelated subqueries:
  - Do not depend on data from the outer query
  - Execute once for the entire outer query
- Correlated subqueries:
  - Make use of data from the outer query
  - Execute once for each row of the outer query
  - Can use the EXISTS operator

Example

Figure 7-8a Processing a noncorrelated subquery



A noncorrelated subquery processes completely before the outer query begins.

CORRELATED SUBQUERY EXAMPLE

Query: What are the order IDs for all orders that have included furniture finished in "Natural Ash" ?

Relational schema

The EXISTS operator returns a TRUE value if the subquery resulted in a non-empty set, otherwise it returns a FALSE

result

```
SELECT DISTINCT OrderID FROM OrderLine_T
WHERE EXISTS
      (SELECT *
         FROM Product _T
              WHERE ProductID = OrderLine_T.ProductID
                    AND Productfinish = 'Natural Ash');
```

- A correlated subquery always refers to an attribute from a table in outer query
- The subquery tests for a value that comes from outer query

CORRELATED SUBQUERY EXAMPLE

Query: Display the order IDs for all orders that have included furniture finished in "Natural Ash" ?

```
SELECT DISTINCT OrderID FROM OrderLine_T
WHERE EXISTS
        (SELECT *
          FROM Product _T
                WHERE ProductID = OrderLine_T.ProductID
                    AND Productfinish = 'Natural Ash');
```

- **The subquery is executed once for each data row in the outer query.**
- **The subquery checks for each order line to see if the finish of the ordered product is natural ash.**
  - **If TRUE, the outer query displays the order ID for that order.**

Detailed steps

Result:

ORDERID
1001
1002
1003
1006
1007
1008
1009

7 rows selected.

Figure 7-8b Processing a correlated subquery

Note: Only the orders that involve products with Natural Ash will be included in the final results.

(b) Processing a correlated subquery

What are the order IDs for all orders that have included furniture finished in natural ash?

```
SELECT DISTINCT OrderID FROM OrderLine_T
WHERE EXISTS
        (SELECT *
          FROM Product _T
                WHERE ProductID = OrderLine_T.ProductID
                    AND Productfinish = 'Natural Ash');
```

Subquery refers to outer-query data, so it executes once for each row of outer query

	OrderID	ProductID	OrderedQuantity
1 →	1001	1	1
	1001	2	2
	1001	4	1
3 →	1002	3	5
	1003	3	3
	1004	6	2
	1004	8	2
	1005	4	4
	1006	4	1
	1006	5	2
	1007	1	3
	1007	2	2
	1008	3	3
	1008	8	3
	1009	4	2
	1009	7	3
	1010	8	10
*	0	0	0

	ProductID	ProductDescription	ProductFinish	ProductStandardPrice	ProductLineID
▶	1	End Table	Cherry	$175.00	10001
2 →	2	Coffee Table	Natural Ash	$200.00	20001
4 →	3	Computer Desk	Natural Ash	$375.00	20001
	4	Entertainment Center	Natural Maple	$650.00	30001
	5	Writer's Desk	Cherry	$325.00	10001
	6	8-Drawer Dresser	White Ash	$750.00	20001
	7	Dining Table	Natural Ash	$800.00	20001
	8	Computer Desk	Walnut	$250.00	30001
*	(AutoNumber)			$0.00	

1. The first order ID is selected from OrderLine_T: OrderID =1001.
2. The subquery is evaluated to see if any product in that order has a natural ash finish. Product 2 does, and is part of the order. EXISTS is valued as true and the order ID is added to the result table.
3. The next order ID is selected from OrderLine_T: OrderID =1002.
4. The subquery is evaluated to see if the product ordered has a natural ash finish. It does. EXISTS is valued as true and the order ID is added to the result table.
5. Processing continues through each order ID. Orders 1004, 1005, and 1010 are not included in the result table because they do not include any furniture with a natural ash finish. The final result table is shown in the text on page 302.

EXAMPLE OF SUBQUERY AS A TA

Relational schema

Query : Show the product description, product standard price, and overall average standard price for all products that have a standard price that is higher than the average standa

Subquery forms a derived table used in the FROM clause of the outer query

One column of the subquery is an aggregate function that has an alias name. That alias can then be referred to in the outer query.

```
SELECT ProductDescription, ProductStandardPrice, AvgPrice
   FROM
      (SELECT AVG(ProductStandardPrice) AvgPrice FROM Product_T),
         Product_T
   WHERE ProductStandardPrice > AvgPrice;
```

The WHERE clause normally cannot include aggregate functions. But, because the aggregate is performed in the subquery, its result is a derived table that can be used in the outer query's WHERE clause.

result

RESULT OF USING SUBQUERY AS A TABLE

```
SELECT ProductDescription, ProductStandardPrice, AvgPrice
  FROM
     (SELECT AVG(ProductStandardPrice) AvgPrice FROM Product_T),
        Product_T
  WHERE ProductStandardPrice > AvgPrice;
```

used

used

Attribute of a table derived from a subquery

Result:

PRODUCTDESCRIPTION	PRODUCTSTANDARDPRICE	AVGPRICE
Entertainment Center	650	440.625
8-Drawer Dresser	750	440.625
Dining Table	800	440.625

UNION QUERIES

Combine the output of multiple queries (union of multiple queries) into a single result table

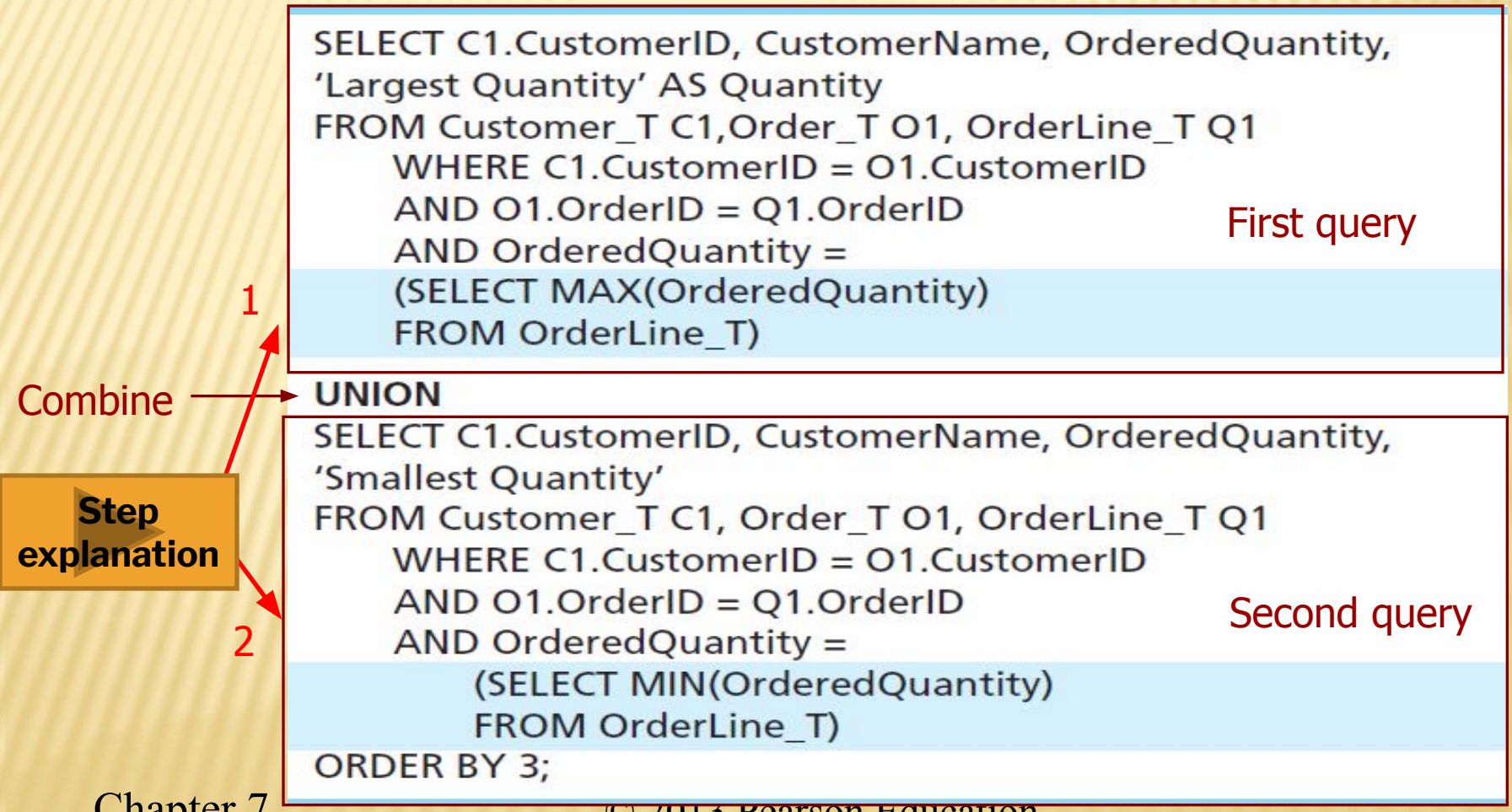


Figure 7-9 Combining queries using UNION

2

```
SELECT C1.CustomerID, CustomerName, OrderedQuantity, 'Largest Quantity' AS Quantity
    FROM Customer_T C1,Order_T O1, OrderLine_T Q1
    WHERE C1.CustomerID = O1.CustomerID
            AND O1.OrderID = Q1.OrderID
            AND OrderedQuantity =
                        (SELECT MAX(OrderedQuantity)
                        FROM OrderLine_T)
```

1

1. In the above query, the subquery is processed first and an intermediate results table created. It contains the maximum quantity ordered from OrderLine_T and has a value of 10.
2. Next the main query selects customer information for the customer or customers who ordered 10 of any item. Contemporary Casuals has ordered 10 of some unspecified item.

```
SELECT C1.CustomerID, CustomerName, OrderedQuantity, 'Smallest Quantity'
    FROM Customer_T C1, Order_T O1, OrderLine_T Q1
    WHERE C1.CustomerID = O1.CustomerID
            AND O1.OrderID = Q1.OrderID
            AND OrderedQuantity =
                        (SELECT MIN(OrderedQuantity)
                         FROM OrderLine_T)
ORDER BY 3;
```

Note: In UNION queries, the number and data types of the attributes in the SELECT clauses of both queries must be identical.

1. In the second main query, the same process is followed but the result returned is for the minimum order quantity.
2. The results of the two queries are joined together using the UNION command.
3. The results are then ordered according to the value in OrderedQuantity. The default is ascending value, so the orders with the smallest quantity, 1, are listed first.

result

RESULT OF UNION EXAMPLE

```
SELECT C1.CustomerID, CustomerName, OrderedQuantity,
'Largest Quantity' AS Quantity
FROM Customer_T C1,Order_T O1, OrderLine_T Q1
      WHERE C1.CustomerID = O1.CustomerID
      AND O1.OrderID = Q1.OrderID
      AND OrderedQuantity =
      (SELECT MAX(OrderedQuantity)
      FROM OrderLine_T)
UNION
SELECT C1.CustomerID, CustomerName, OrderedQuantity,
'Smallest Quantity'
FROM Customer_T C1, Order_T O1, OrderLine_T Q1
      WHERE C1.CustomerID = O1.CustomerID
      AND O1.OrderID = Q1.OrderID
      AND OrderedQuantity =
            (SELECT MIN(OrderedQuantity)
            FROM OrderLine_T)
ORDER BY 3;
```

Result:

CUSTOMERID	CUSTOMERNAME	ORDEREDQUANTITY	QUANTITY
1	Contemporary Casuals	1	Smallest Quantity
2	Value Furniture	1	Smallest Quantity
1	Contemporary Casuals	10	Largest Quantity

Data

TIPS FOR DEVELOPING QUERIES

- Be familiar with the data model (entities and relationships) or relational schema
- Understand the data and desired results
- Know the attributes desired in results
- Identify the entities that contain desired attributes
- Review ERD
- Construct a WHERE equality for each link
- Fine tune with GROUP BY and HAVING clauses if needed
- Consider the effect on unusual data

D.M.

R.S.

data

QUERY EFFICIENCY CONSIDERATIONS

- Instead of SELECT *, identify the specific attributes in the SELECT clause; this helps reduce network traffic of result set
- Limit the number of subqueries; try to make everything done in a single query if possible
- If data is to be used many times, make a separate query and store it as a view

GUIDELINES FOR BETTER QUERY DESIGN

- Understand how indexes are used in query processing
- Keep optimizer statistics up-to-date
- Use compatible data types for fields and literals
- Write simple queries
- Break complex queries into multiple simple parts
- Don’t nest one query inside another query if possible
- Don’t combine a query with itself (if possible avoid self-joins)

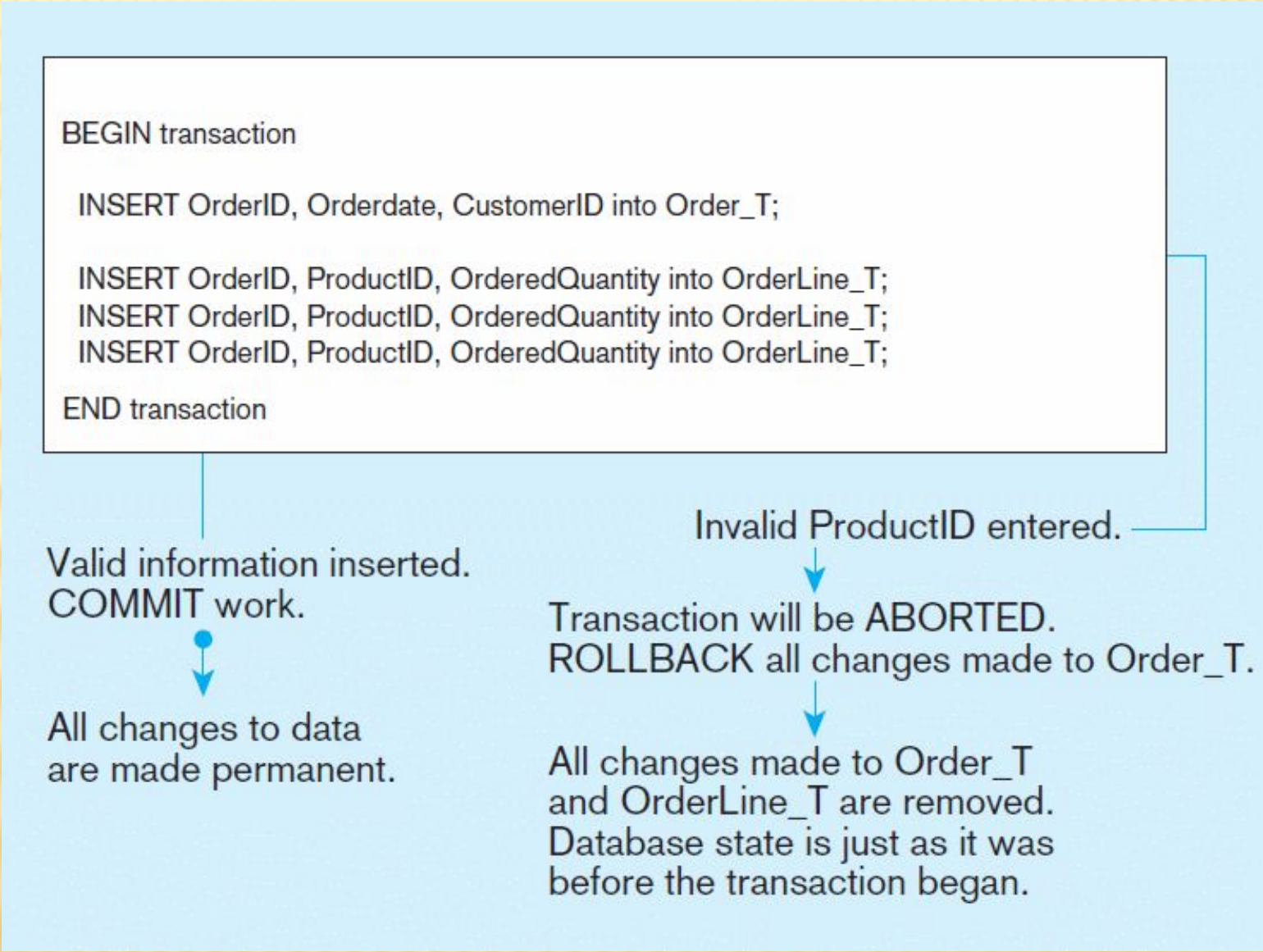
GUIDELINES FOR BETTER QUERY DESIGN (CONT.)

- Create temporary tables for groups of queries
- Combine multiple update commands into one if possible
- Retrieve only the data you need
- Don’t have the DBMS sort without an index
- Practice more!
- Consider the total query processing time for ad hoc queries

ENSURING TRANSACTION INTEGRITY

- Transaction = A set of works that must be completely processed or not processed at all
  - May involve multiple updates
  - If any update fails, then all other updates must be cancelled
- SQL commands for transactions
  - BEGIN TRANSACTION/END TRANSACTION
    - Marks boundaries of a transaction
  - COMMIT
    - Makes all updates permanent
  - ROLLBACK
    - Cancels updates since the last COMMIT

Figure 7-12 An SQL Transaction example (in pseudocode)



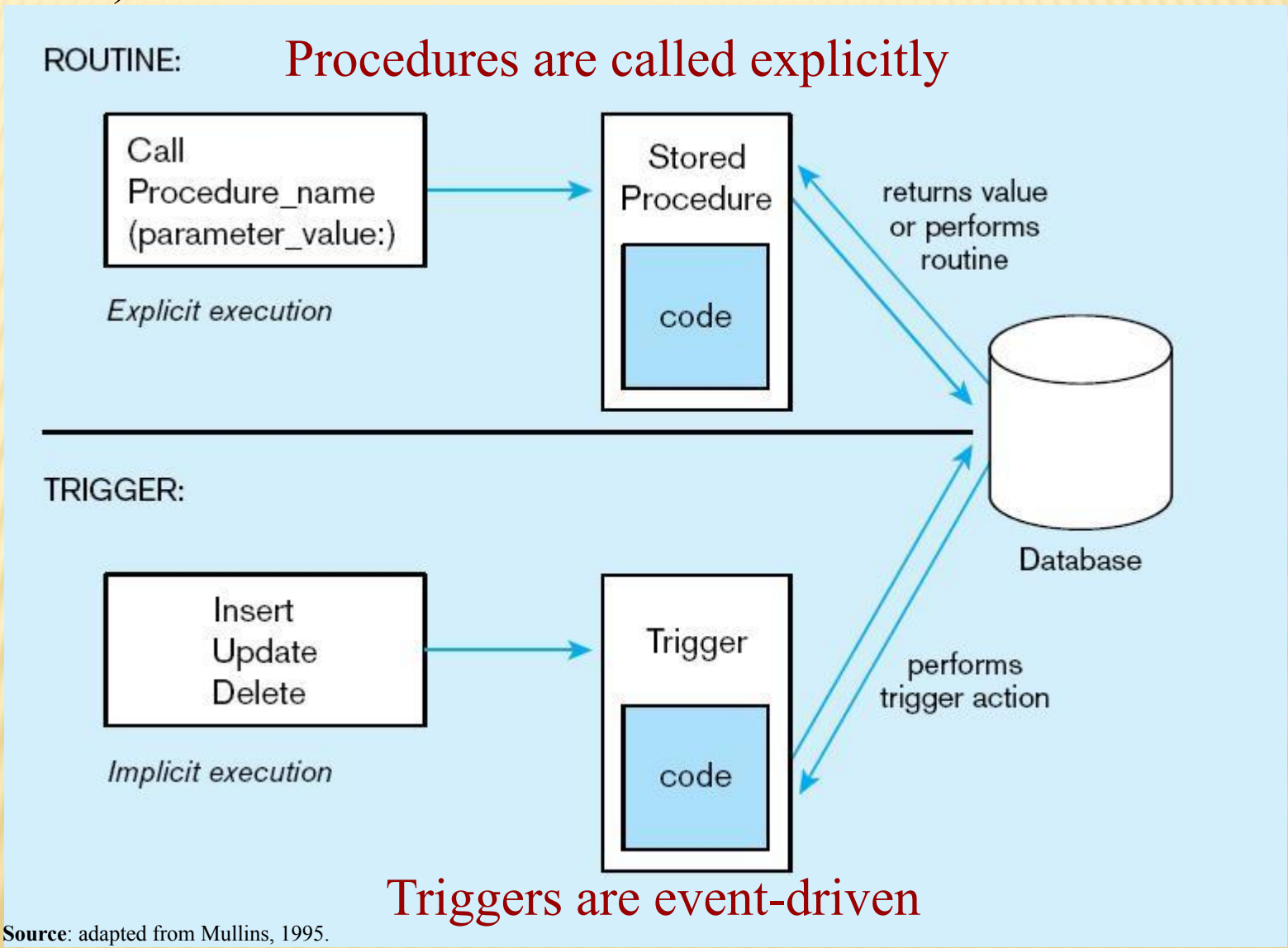
DATA DICTIONARY FACILITIES

- System tables that store metadata
- Users usually can view some of these tables
- Users are restricted from updating them
- Some examples in Oracle 11g
  - DBA_TABLES – descriptions of tables
  - DBA_CONSTRAINTS – description of constraints
  - DBA_USERS – information about the users
- Examples in Microsoft SQL Server 2008
  - sys.columns – table and column definitions
  - sys.indexes – table index information
  - sys.foreign_key_columns – details about columns in foreign key constraints

ROUTINES AND TRIGGERS

- **Routines**
  - Program modules that execute on demand
- **Functions**–routines that return values and take input parameters
- **Procedures**–routines that do not return values and can take input or output parameters
- **Triggers**–routines that execute in response to a database event (INSERT, UPDATE, or DELETE)

Figure7-13 Triggers contrasted with stored procedures (based on Mullins 1995)



Source: adapted from Mullins, 1995.

Figure 7-14 Simplified trigger syntax, SQL:2008

```
CREATE TRIGGER trigger_name
        {BEFORE | AFTER | INSTEAD OF} {INSERT | DELETE | UPDATE} ON
        table_name
        [FOR EACH {ROW | STATEMENT}] [WHEN (search condition)]
        <triggered SQL statement here>;
```

EMBEDDED AND DYNAMIC SQL

- Embedded SQL
  - SQL statements hard-coded in a program written in another language such as C or Java
- Dynamic SQL
  - SQL code dynamically generated by an application program, as the application is running